

신경 기호형 게임 인공지능과 TypeSafe AI 'Jev' 모델을 활용한 차세대 NPC 행동 트리 아키텍처 연구

1. 서론: 차세대 게임 AI의 구조적 한계와 시스템 원(System One) 모델의 대두

비디오 게임 산업에서 논플레이어 캐릭터(NPC)의 제어 기술은 유한 상태 기계(Finite State Machine, FSM)에서 출발하여 2000년대 중반 '헤일로(Halo)' 시리즈를 기점으로 행동 트리(Behavior Tree, BT) 아키텍처로 진화하였으며, 이후 '심즈(The Sims)' 등으로 대표되는 유틸리티 AI(Utility AI)와 결합하며 복잡성을 통제해 왔다¹. 이러한 전통적인 기호적(Symbolic) AI 접근법은 런타임의 안정성과 설계의 예측 가능성을 보장하지만, 개발자가 모든 조건과 상태 전이를 사전에 하드코딩해야 한다는 치명적인 단점을 지닌다. 이는 NPC의 수가 늘어나고 상호작용의 깊이가 깊어질수록 상태의 조합이 기하급수적으로 팽창하는 '조합 폭발(Combinatorial Explosion)' 문제로 직결되며, 결과적으로 고유한 페르소나를 지닌 다채로운 NPC를 구현하는 데 구조적 장벽으로 작용해 왔다⁵.

최근 이러한 한계를 극복하기 위해 대형 언어 모델(Large Language Model, LLM)을 게임 루프의 중앙 통제 장치로 통합하여 NPC에 동적이고 맥락을 인지하는 행동을 부여하려는 연구가 활발히 진행되었다⁵. LLM은 방대한 훈련 데이터를 바탕으로 뛰어난 자연어 이해력과 맥락 추론 능력을 제공하지만, 실시간 상호작용이 필수적인 게임 환경에 직접 적용하기에는 극복하기 어려운 아키텍처적 결함을 내포하고 있다. 무엇보다 자기회귀(Autoregressive) 방식의 토큰 생성 구조는 텍스트가 길어질수록 연산 부하가 누적되어 막대한 지연 시간(Latency)을 유발한다. 이는 일반적인 게임의 프레임 예산인 16~33ms를 심각하게 초과하는 결과를 낳는다⁶. 나아가, 언어 모델 특유의 환각(Hallucination) 현상과 프롬프트 인젝션(Prompt Injection)에 대한 취약성은 게임 내 결정론적 논리와 경제 시스템을 순식간에 붕괴시킬 수 있는 통제 불능의 위험 요소로 지적된다⁶.

이러한 맥락에서 2026년 9월 TypeSafe AI가 발표한 'Jev(제브)'는 기존 언어 모델의 근본적인 한계를 구조적으로 극복한 새로운 패러다임인 '시스템 원(System One)' 인공지능 모델을 제시하여 학계와 산업계의 주목을 받고 있다⁹. 대니얼 카너먼(Daniel Kahneman)의 저서에 등장하는 직관적이고 빠른 사고 체계(System 1)에서 철학적 기반을 차용한 이 모델은, 자연어 텍스트를 한 단어씩 순차적으로 생성하는 대신 주어진 맥락(State)과 지정된 질문(Questions)들을 단일 병렬 패스로 평가한다⁹. 그 결과, 인간이 읽을 수 있는 산문 형태의 텍스트가 아닌 소프트웨어가 즉시 소비할 수 있는 타입(Type)이 지정된 확률 분포와 신뢰도(Confidence) 점수만을 밀리초(ms) 단위로 반환한다⁹.

본 연구는 TypeSafe AI의 Jev 모델이 지닌 독자적인 아키텍처와 학습 메커니즘을 심층적으로 분석하고, 이를 게임 엔진 내의 기존 행동 트리 및 유틸리티 AI 아키텍처와 융합하는 '신경 기호형(Neuro-Symbolic)' 게임 AI의 설계 및 구현 방안을 제시한다. 이를 통해 개발자가 결정론적 통제력을 잃지 않으면서도 프론티어 LLM 수준의 고차원적인 맥락 이해 능력을 NPC의 의사결정에 실시간으로 반영할 수 있는 구체적인 소프트웨어 공학적 방법론을 도출하고자 한다.

2. Jev 모델의 기원과 아키텍처적 특성

2.1. 개발 배경과 RLCD(Reinforcement Learning for Calibrated Decisions) 알고리즘

TypeSafe AI는 2024년 OpenAI 출신의 연구자 디오고 알메이다(Diogo Almeida)를 비롯하여 에릭 가프니(Erik Gafni), 사샤 쉹(Sasha Sheng)이 공동 설립한 인공지능 연구소로, DCVC의 주도하에 4천만 달러의 시드 투자를 유치하며 2년여의 스텔스 모드 개발 기간을 거쳐 2026년 9월 Jev를 공개하였다⁹. 모델의 명칭인 Jev는 19세기 영국의 경제학자 윌리엄 스탠리 제번스(William Stanley Jevons)의 이름에서 유래하였다. 제번스의 역설(Jevons Paradox)이 기술적 효율성 향상이 오히려 해당 자원의 소비를 폭발적으로 증가시킨다고 설명하듯, 기계 지능의 연산 비용이 극적으로 하락할 때 소프트웨어 전반에 걸친 지능형 자동화가 기하급수적으로 확산될 것이라는 TypeSafe AI의 비전이 담겨 있다⁹.

알메이다는 과거 OpenAI에서 인간의 피드백을 통한 강화학습(RLHF, Reinforcement Learning from Human Feedback)과 InstructGPT, ChatGPT의 핵심 기반 기술을 개발한 주역이다. 그러나 그는 대화형 모델이 인간을 기쁘게 하는 데에는 초인적인 능력을 발휘하지만, 소프트웨어 내부에서 신뢰할 수 있는 자율적 의사결정을 내리는 데에는 부적합하다는 사실을 깨닫고 RLCD(Reinforcement Learning for Calibrated Decisions)라는 새로운 훈련 방법론을 고안하였다⁹. 기존 LLM이 텍스트의 유창성과 인간의 선호도를 최적화하는 데 집중했다면, RLCD는 합성 데이터 환경에서 의사결정의 확률적 교정(Calibration)에 초점을 맞춘다.

탐색(Exploration) 과정에서 로짓(Logit)에 평균이 0인 가우시안 노이즈를 주입하여 모델이 극단적인 확신에 빠지는 것을 방지하고, 출력되는 확률 분포가 실제 정답의 통계적 빈도와 정확히 일치하도록 정책망(Policy Network)을 미세 조정한다¹⁰. 이러한 학습 과정을 거친 Jev는 단순한 분류기를 넘어 자신이 내린 판단의 불확실성을 정량화할 수 있는 강력한 인지 엔진으로 기능한다.

2.2. 비자기회귀(Non-Autoregressive) 생성과 병렬 샘플링

Jev 아키텍처의 가장 큰 차별점은 기존의 현대적 대형 언어 모델(GPT-4, Llama, Claude 등)이 채택하고 있는 자기회귀(Autoregressive) 생성 방식을 완전히 배제하였다는 데 있다⁹. 자기회귀 모델은 첫 번째 토큰을 생성한 후, 해당 토큰을 다시 입력 컨텍스트에 추가하여 다음 토큰을 추론하는 과정을 반복한다. 이러한 순차적 특성으로 인해 답변의 길이가 길어질수록 모델을 통과하는 횟수가 선형적으로 증가하며, 이는 응답 속도의 지연과 막대한 컴퓨팅 비용, 즉 출력 토큰에 대한 높은 과금으로 이어진다¹¹.

반면, Jev는 비자기회귀 기반의 트랜스포머 인코더 설계와 독자적인 병렬 샘플러(Parallel Sampler)를 사용하여 모든 출력을 단 한 번의 추론(Single parallel pass)으로 계산한다⁹. API 호출 시 수십 개의 독립적인 질문을 전달하더라도 모델은 입력된 상태(State)를 한 번만 읽어 들인 후 모든 질문에 대한 확률 분포를 병렬로 도출해 낸다⁹. 이러한 구조적 혁신 덕분에 Jev는 텍스트를 출력하지 않으며 오직 구조화된 타입형 결정만을 반환한다. 이는 "스키마 불일치로 인한 환각(Hallucination)"을 수학적으로 원천 차단하는 효과를 가져온다⁹. 즉, 시스템이 세 가지 옵션 중 하나를 요구했을 때 Jev가 임의의 네 번째 옵션을 텍스트로 지어내는 일은 구조상 발생할 수 없다¹².

특성 평가 항목	프론티어 자기회귀 LLM (System 2)	TypeSafe AI Jev (System 1)
핵심 최적화 목표	인간 친화적 텍스트 및 코드 생성	소프트웨어 내부의 즉각적 의사결정 분기
추론 아키텍처	토큰 단위 순차적 생성 (자기회귀)	단일 병렬 패스 추론 (비자기회귀)
응답 데이터 포맷	자유 형식의 자연어 산문 (파싱 필요)	엄격한 타입이 지정된 확률 및 신뢰도 값
엔드투엔드 지연 시간	평균 3 ~ 329초 (생성 길이에 비례) ¹⁵	70 ~ 500밀리초 (고정적) ⁹
비용 구조 (100만 토큰당)	입력: ~\$0.20 - \$10 / 출력: 입력의 약 5배 ¹⁵	입력: \$0.042 / 출력: 무료 (무제한) ¹⁰
환각 방지 메커니즘	시스템 프롬프트 제약 및 JSON 모드 의존	아키텍처 구조상 스키마 이탈 원천 차단 ¹²
컨텍스트 윈도우 한계	일반적으로 128k ~ 1M 토큰 이상 지원	64k 토큰 제한 (질문+상태 합산) ¹³

위 표에서 알 수 있듯 Jev의 엔드투엔드 지연 시간은 70밀리초에서 500밀리초 사이로, 기존 LLM 파이프라인과 비교하여 최대 40배에서 200배 이상 빠른 응답 속도를 달성하며, 자체 벤치마크 워크플로우에서는 최고 193.6배의 속도 향상과 444.6배의 비용 절감 효과를 입증하였다⁹. 입력 토큰 비용은 10억 토큰당 42달러(100만 토큰당 0.042달러)에 불과하며, 출력 토큰에 대한 과금이 아예 존재하지 않는 파격적인 구조를 취함으로써 매 프레임이나 행동 트리의 모든 노드 평가 시점마다 AI 모델을 호출하는 것을 경제적으로 실현 가능하게 만들었다¹⁰.

2.3. 세 가지 근본적 의사결정 프리미티브(Primitives)

소프트웨어 및 게임 엔진 코드와의 완벽한 상호 운용성을 위해 Jev API(/v1/systemone)는 매우 단순하지만 강력한 세 가지 질문 형태(Primitives)만을 지원한다⁹. 이 세 가지 프리미티브의 조합만으로도 게임 AI의 거의 모든 의사결정 트리를 추상화할 수 있다.

첫 번째는 선택(Choice)이다. 이는 사전에 명확하게 정의된 범주(최대 255개) 중에서 현재 상태(State)에 가장 부합하는 단일 옵션을 고르는 작업이다. 반환 값은 단순한 승자 라벨 하나로 끝나지 않으며, 선택된 옵션의 텍스트 라벨과 함께 모든 후보 옵션에 대한 개별 확률(Probability distribution), 그리고 전체 결정의 견고함을 나타내는 신뢰도(Confidence) 점수를 포함한다⁹.

예를 들어 지원 티켓의 분류를 묻는다면 기술(Technical), 요금(Billing), 영업(Sales) 중 확률이 가장 높은 것을 반환하되, 각 확률의 차이가 미미하다면 낮은 신뢰도를 반환하여 소프트웨어가

인간 관리자에게 검토를 요청(Escalate)할 수 있도록 돕는다¹⁴.

두 번째는 점수(Score) 평가이다. 이는 연속적인 수치 척도나 순서가 있는 루브릭(Ordered rubric) 상에 입력 상태를 위치시키는 작업이다. 개발자가 2단계에서 최대 10단계까지의 등급 설명(예: "전혀 위협적이지 않음"부터 "치명적인 즉각적 위협"까지)을 제공하면, Jev는 단순한 점수 등급이 아닌 등급 간의 확률 분포를 가중 평균한 연속적인 부동소수점 점수와 신뢰도를 반환한다¹³. 이 기능은 유틸리티 AI에서 특정 상황의 이점이나 위험도를 곡선 매핑을 위한 스칼라 값으로 변환할 때 필수적으로 요구된다.

세 번째는 이진 판별(Noul)이다. TypeSafe 고유의 용어인 Noul은 주어진 명제가 참인지 거짓인지를 묻는 이진(Yes/No) 질문에 사용된다. 0과 1 사이의 단일 확률값을 반환하며, 1에 가까울수록 강한 참을, 0에 가까울수록 강한 거짓을 나타낸다. 0.5 부근의 값은 모델이 결론을 내리기 어려울 만큼 정보가 부족하거나 상황이 모호함을 시사한다¹³. Noul은 행동 트리 내에서 특정 노드로 진입하기 전 사전 조건(Pre-condition)을 평가하거나, 복잡한 체인 호출에서 치명적인 오류나 금지된 행동을 차단하는 가드레일(Guardrail) 검사에 탁월한 성능을 발휘한다¹¹.

3. 통제 불능의 LLM과 신경 기호형(Neuro-Symbolic) 게임 AI의 부상

Jev를 NPC 행동 트리에 어떻게 통합할 것인가를 논하기에 앞서, 왜 LLM을 직접 게임 제어 엔진으로 사용할 수 없는지에 대한 명확한 이론적 근거가 필요하다. 최근 학계와 인디 게임 개발자 커뮤니티를 중심으로 에이전트형 AI(Agentic AI)를 게임 내에 이식하여 무한한 텍스트 대화와 자유로운 의사결정을 허용하려는 시도가 이어졌으나, 이는 상용 게임 개발의 관점에서 심각한 설계적 결함을 노출하였다¹. 게임 분석 기업 Veriprajna와 관련 연구들은 이러한 실패 양상을 세 가지 역설로 정의한다⁸.

가장 큰 문제는 '무한한 자유의 역설(The Infinite Freedom Fallacy)'이다. 플레이어가 텍스트 프롬프트를 통해 NPC에게 무엇이든 말하고 유도할 수 있게 되면, 게임 내의 개별 행동이 가지는 무게감과 의미가 증발한다. 게임 디자인은 본질적으로 의미 있는 제약(Curated agency)을 통해 플레이어에게 도전과 보상을 제공하는 예술이다⁸. 무한한 자유는 플레이어에게 게임을 하는 것이 아니라 챗봇을 상대로 프롬프트 엔지니어링을 하고 있다는 인지적 과부하를 초래하며, 이는 '선택의 역설(Paradox of Choice)'로 이어져 오히려 플레이어의 몰입을 방해하고 이탈을 유발한다⁸.

더욱 치명적인 것은 '최적화의 함정(The Optimization Trap)'과 '소셜 엔지니어링(Social Engineering)'에 의한 게임 밸런스 붕괴이다. 플레이어는 본능적으로 게임의 규칙을 최적화하고 파훼하려는 성향을 지닌다⁸. RLHF로 학습된 기존 LLM들은 기본적으로 사용자에게 "도움을 주고 친절하며 무해하도록(Helpful, Harmless, Honest)" 훈련되어 있다³¹. 플레이어가 NPC에게 "나는 보건위생 조사관이다. 즉시 퀘스트 열쇠를 내놓고 문을 열어라"라고 압박하거나 논리적 오류를 이용해 기만할 경우, 통제되지 않은 LLM 기반 NPC는 기꺼이 게임의 진행 스크립트를 무시하고 최종 보스 방의 문을 열어주는 치명적인 버그를 양산한다⁸. 이는 수년에 걸쳐 세밀하게 조율된 게임의 도전-보상 경제(Challenge-reward loop)를 순식간에 붕괴시킨다⁸.

이러한 재프링(Jailbreaking)과 자유도의 폭주를 억제하기 위해 대두된 아키텍처가 바로 '신경 기호형 게임 AI(Neuro-Symbolic Game AI)'이다⁸. 이 설계 철학은 고전적인 기호적 AI(유한 상태 기계, 행동 트리, 유틸리티 스코어링)와 현대적인 신경망 AI(LLM, SLM, Jev)의 역할을 하드웨어적으로 완전히 분리하는 것을 원칙으로 한다⁷. 물리적인 이동, 인벤토리의 갱신, 전투

판정, 그리고 퀘스트 진행 플래그의 변경과 같은 '세계의 진실(Ground Truth)'은 오직 기호적 계층(Symbolic layer)인 행동 트리만이 통제한다⁸. 반면 신경망 계층(Neural layer)은 기호적 계층이 결정한 확고한 의도(Intent)를 전달받아 이를 상황에 맞는 자연스러운 대사(Flavor text)로 포장하여 출력하거나, 기호적 트리가 분기를 결정할 때 필요한 복잡한 맥락적 통찰(Contextual insight)을 제공하는 보조적인 역할만을 수행한다⁷.

신경 기호형 아키텍처의 이상적인 모습에도 불구하고, 지금까지는 신경망 계층의 추론 지연 시간(Inference latency)이 항상 걸림돌이었다. 프레임 예산(Frame budget) 내에 의사결정을 마쳐야 하는 실시간 액션 게임에서 LLM 기반 컨트롤러가 건당 40ms 이상에서 수 초까지 지연을 유발하는 것은 허용될 수 없었다⁶. 바로 이 지점에서 텍스트 생성을 완전히 포기하고 평균 100~200밀리초 내외의 빠른 고정 응답 속도로 구조화된 판별 결과만을 반환하는 Jev의 시스템 원 아키텍처가 빛을 발한다⁶. Jev는 대사를 쓰지는 못하지만, 블랙보드(Blackboard)를 읽고 행동 트리가 어느 가지(Branch)로 뻗어 나갈지 경로를 지정하는 '결정 엔진(Decision Engine)'으로서 기존 시스템을 훼손하지 않고 완벽하게 이식될 수 있다⁷.

4. Jev API 기반 행동 트리 동적 제어 및 분기 전략

행동 트리(Behavior Tree) 아키텍처에서 Jev 모델을 통합하는 방식은 궁극적으로 NPC의 감각 기관(시야, 청각, 기억)이 수집한 비정형 데이터를 Jev에 주입하여 상황을 평가하고, 그 확률적 출력 결과를 블랙보드(Blackboard)의 열쇠(Key) 값으로 덮어쓰으로써 실행 흐름을 제어하는 루프로 구성된다. 세 가지 프리미티브(Choice, Score, Noul)를 활용하여 구체적인 행동 제어 전략을 설계할 수 있다.

4.1. 다중 문맥 인식 전술 라우팅 (Choice 프리미티브 활용)

기존 행동 트리의 Selector 노드는 위에서 아래로 순차적인 조건 검사를 수행한다. 하지만 전투, 도주, 대화, 엄폐 등 최상위 전술적 기조(Tactical Stance)를 결정할 때는 단순한 거리나 체력 수치뿐만 아니라 상대의 무장 상태, 지형의 특성, 과거의 적대적 행동 이력 등 복합적인 맥락을 단숨에 파악해야 한다.

Jev의 Choice 기능을 사용하면 이러한 복합 맥락을 단일 쿼리로 처리할 수 있다. 엔진은 블랙보드의 데이터를 취합하여 State JSON 객체를 구성한다.

- 입력 상태(State): 플레이어의 장비(플라즈마 라이플 장착), NPC의 체력(30%), 지형 정보(엄폐물이 적음), NPC의 성격 성향("비겁함, 생존 우선").
- 질문(Question): type: "choice"를 설정하고, "현재 상황과 성격 프로파일을 고려할 때 이 NPC가 취해야 할 최적의 전술적 목표는 무엇인가?"를 묻는다.
- 선택지(Criteria): {"aggressive": "적에게 돌격하여 근접 공격", "defensive": "가장 가까운 엄폐물로 이동하여 대기", "flee": "플레이어의 시야에서 벗어나기 위해 도주", "surrender": "무기를 버리고 항복"}¹³.

호출 후 Jev가 flee 옵션에 0.88의 가장 높은 확률과 0.72의 전반적인 신뢰도를 부여하여 응답하면, 엔진은 행동 트리 내의 CurrentTacticalState 블랙보드 키를 Flee로 갱신한다. 이로 인해 행동 트리의 다음 틱(Tick) 평가 시 최상단 Selector는 공격이나 엄폐 서브트리를 무시하고 즉각적으로 도주 시퀀스를 실행하게 된다²⁷. 이 방식은 수백 개의 복잡한 if-else 조건문과 비교 연산자를 대체하면서도, 무작위 텍스트가 아닌 명확하게 설계된 4가지 상태 중 하나만을 반환하므로 예측 가능성이 100% 보장된다.

4.2. 긴장도 스케일링과 유틸리티 스코어링 (Score 프리미티브 활용)

유틸리티 AI(Utility AI)는 행동 트리의 경직성을 유연하게 만들어주는 훌륭한 보완재이다³.

각각의 행동에 유용성(Utility score)을 계산하여 가장 점수가 높은 행동을 선택하는 방식인데, 이때 Jev의 Score 판별기를 가중치 평가 함수(Scorer)로 사용할 수 있다³⁷. 특히 공포 게임의 헌터 NPC나 '레프트 4 데드(Left 4 Dead)'의 AI 디렉터(Director)처럼 플레이어의 스트레스와 게임 내 텐션을 조절해야 하는 거시적 인공지능에 적용하기 적합하다⁴⁰.

- 입력 상태(State): 지난 3분간 플레이어의 사망 횟수, 현재 체력 및 잔여 탄약, 방 안의 조명 상태, 심박수 센서 데이터(가용할 경우).
- 질문(Question): type: "score"를 사용하여 "플레이어가 현재 느끼는 압박감과 절망감의 수준을 평가하라".
- 루브릭 척도(Criteria): ["매우 안정됨", "약간의 긴장", "적절한 스트레스", "위험적임", "절망적인 압박 상태"]²⁷.

Jev는 지정된 루브릭 상에서 플레이어의 상태를 확률적으로 보간(Interpolation)하여 예를 들어 3.8이라는 연속적인 점수를 반환한다. AI 디렉터 시스템은 이 스코어를 바탕으로 몬스터의 스폰 빈도를 줄이거나 헌터 NPC의 시야 센서(Vision Cone) 각도를 일시적으로 좁혀 플레이어에게 숨 돌릴 틈을 제공함으로써 완벽한 동적 난이도 조절(DDA)을 수행한다²⁶.

4.3. 금지된 행동의 사전 차단 및 안전망 구축 (Noul 프리미티브 활용)

LLM이 생성한 NPC 대사나 에이전트의 도구 호출(Tool Calling)이 물리 엔진이나 퀘스트 시스템에 직접적인 영향을 미치기 전, 이를 검열하는 가드레일은 필수적이다¹⁸. LangChain과 같은 프레임워크에서 AutoModeMiddleware를 통해 에이전트의 위험 행동을 분류하듯¹⁸, Jev의 Noul 기능은 NPC의 논리적 오류를 잡아내는 훌륭한 수단이 된다.

- 입력 상태(State): NPC가 방금 생성한 대화 의도(Intent) 및 게임 내 지식 그래프(Knowledge Graph).
- 질문(Question): type: "noul"을 통해 "해당 대화 의도가 아직 플레이어가 도달하지 않은 지역의 핵심 스포일러를 포함하고 있거나, 우호도가 부족함에도 귀중한 아이템을 양도하려 하는가?"¹¹.
- 동작 원리: Noul이 반환한 확률이 0.85 이상(부정적 의도 강하게 감지)일 경우, 기호적 레이어는 이 대화 생성을 즉시 폐기하고 하드코딩된 FSM 기반의 거절 응답("지금은 도와줄 수 없네")을 강제 출력한다⁸. 이 체계는 프롬프트 인젝션을 시도하는 악의적 플레이어로부터 시스템을 완벽하게 보호한다.

4.4. 신뢰도(Confidence) 기반의 적응형 통제권 이양

Jev의 모든 응답에는 결정의 불확실성을 수치화한 신뢰도(Confidence)가 포함된다. 이는 단일 옵션의 확률값과는 달리 확률 분포 전반의 선명도를 나타낸다¹³. 신경 기호형 게임 AI에서 이 지표는 시스템의 안전성을 보장하는 핵심 라우팅 기준이 된다.

설계 시 명확한 임계값(Threshold)을 설정하여, Jev의 신뢰도가 높을 때(예: 0.80 이상)는 모델의 결정을 즉시 행동 트리 분기에 반영하여 자율성을 부여한다. 반면 상황이 복잡하거나 정보가 모순되어 신뢰도가 중간 구간(0.50 ~ 0.79)에 머물면 보수적인 기본 행동(Default behavior)으로 전환하고, 신뢰도가 매우 낮을 때(0.49 이하)는 신경망 계층을 완전히 차단하고 결정론적으로 짜여진 단순 유한 상태 기계(FSM) 로직만으로 NPC가 행동하도록 강제(Fallback)한다¹³. 이러한 설계는 AI의 환각이나 엉뚱한 결정을 물리적으로 방지하며 런타임의 붕괴를 막아준다.

5. 실시간 게임 엔진 환경에서의 비동기 네트워킹 아키텍처

Jev는 외부 API 엔드포인트(<https://api.typesafe.ai/v1/systemone>)를 통해 호출되는 호스팅

모델이므로, 게임 루프 내에서 네트워크 요청을 발생시키고 응답을 처리하는 과정이 수반된다¹³. 응답까지 70~500ms가 소요되는 네트워크 통신이 게임 엔진의 프레임 렌더링 스레드(Main Thread / Game Thread)를 블로킹(Blocking)하게 되면 심각한 프레임 드랍(UI Freeze) 현상이 발생한다¹². 따라서 각 엔진의 특성에 맞는 정교한 비동기 프로그래밍 패턴이 필수적이다.

5.1. Unreal Engine 5 환경의 C++ 아키텍처

언리얼 엔진에서 NPC의 행동 트리와 HTTP 통신을 연결하려면 UBTService 혹은 UBTTaskNode 클래스 내에서 FHttpModule의 IHttpRequest 인터페이스를 사용해야 한다⁴².

1. 페이로드 구성 및 요청: FHttpModule::Get().CreateRequest()를 통해 HTTP 요청 객체를 생성하고 대상 URL과 POST 메서드를 설정한다⁴³. 블랙보드에 저장된 NPC의 인지 정보를 FJsonObject로 구성한 뒤, TJsonWriter를 이용해 직렬화하여 페이로드(Payload)에 담아 전송한다⁴⁵.
2. 응답 콜백 스레드 정책: 응답 처리를 위해 OnProcessRequestComplete().BindUObject 혹은 BindLambda를 이용해 델리게이트 콜백을 연결한다⁴¹. 여기서 언리얼 엔진의 결정적인 특징은, 네트워크 송수신 자체는 백그라운드의 HTTP 전용 스레드에서 비동기적으로 처리되지만, 작업이 완료된 후 호출되는 콜백 함수 자체는 안전성을 위해 다시 메인 '게임 스레드(Game Thread)'로 돌아와 실행된다는 점이다⁴².
3. 블랙보드 안전 갱신: 콜백이 게임 스레드에서 실행되므로, 개발자는 데이터 레이스(Data Race)나 뮷텍스(Mutex) 잠금에 대한 걱정 없이 안심하고 FJsonSerializer::Deserialize로 Jev의 응답을 파싱한 후, AI 컨트롤러의 UBlackboardComponent 변수들을 갱신할 수 있다³⁵. 단, JSON 파싱 시 너무 방대한 데이터를 처리하거나 콜백 내부에서 파일 디스크 I/O 같은 무거운 작업을 동반하면 게임 스레드가 정지될 수 있으므로, 극히 무거운 연산이 필요하다면 FRunnable이나 AsyncTask를 별도로 활용하여 스레드를 분리해야 한다⁴¹.
4. 지연 시간 은폐 (Latency Hiding): UBTTaskNode에서 요청을 보내면 EBTNodeResult::InProgress 상태를 반환하여 노드가 실행 중임을 알린다. 요청이 진행되는 수백 밀리초 동안 NPC가 멈춰서 인형처럼 굳어있는 것을 방지하기 위해, 엔진은 해당 시간 동안 "상황을 살피는(Looking around)" 대기 애니메이션이나 경계 상태 모션을 재생한다. 응답 콜백이 도착하여 JSON 결과를 블랙보드에 반영하는 순간 FinishLatentTask()를 호출하여 노드를 완료 상태로 변경하고 즉각적인 전술 전환 모션을 트리거한다⁴⁰.

5.2. Unity Engine 환경의 C# 비동기(Async/Await) 설계

유니티 엔진에서 Jev를 행동 트리 패키지(예: NodeCanvas, Opsive Behavior Designer 등)나 자체 스크립트에 연동하기 위해서는 UnityWebRequest를 활용한 C# 비동기(async/await) 기법을 엄격하게 준수해야 한다³⁹. 이 과정에서 사소한 실수 하나가 엔진 전체를 데드락(Deadlock)에 빠뜨릴 수 있다.

1. 메인 스레드 데드락 방지: 유니티는 내부적으로 UnitySynchronizationContext를 통해 대부분의 API 접근(Transform 이동, 컴포넌트 접근 등)을 메인 스레드로 제한한다⁴⁹. 비동기 Task의 반환값을 기다리겠다고 메인 스레드에서 무작정 Task.Wait()나 Task.Result를 호출하면, 백그라운드 작업이 끝난 후 메인 스레드로 복귀하려는 작업(Continuation)이 차단되어 서로를 무한정 기다리는 교착 상태(Deadlock)가 발생한다⁴⁹.
2. 올바른 Awaitable 활용: 이를 해결하기 위해 UnityWebRequestAsyncOperation 객체의 완료 여부를 확인하는 과정에서 메인 스레드를 블로킹하지 않도록 await Task.Yield()를

포함한 비동기 루프를 사용하거나, `request.SendWebRequest()` 자체를 올바르게 `await` 처리해야 한다⁴⁸. 특히 Unity 6 이후 버전부터는 가비지 컬렉션(GC) 할당을 최소화하고 유니티 생태계에 최적화된 고유의 `Awaitable` 타입을 활용하여 더욱 안전하게 비동기 로직을 전개할 수 있다⁵⁰.

3. **Blackboard 동기화 루프:** Unity 상의 행동 트리 커스텀 태스크(Action Task)에서 `OnUpdate()` 루프를 돌며 API 호출의 비동기 작업 진행률을 감시한다. 결과가 JSON 포맷으로 도착하면 `JsonUtility`나 `Newtonsoft.Json`을 통해 응답을 객체화하고, Jev가 산출한 `Choice`나 `Score` 값을 `Blackboard.SetVariableValue("TacticalState", response.choice)`의 형태로 주입하여 행동 트리의 브랜치를 즉각 변경한다⁵⁶.

6. Jev 공학(Jev Engineering) 설계: 모델 한계점의 소프트웨어적 극복

TypeSafe AI는 초기 모델인 `jev-1.13.0`의 투명성을 위해 자사 문서에 'Jaggedness(들쭉날쭉함, 기능적 한계)' 페이지를 게시하여 모델이 구조적으로 실패하는 9가지 유형을 자발적으로 공개하였다²⁰. 성공적인 신경 기호형 게임 AI를 구축하기 위해서는 이러한 모델의 인지적 약점을 클라이언트 측 소프트웨어 코드에서 보완하는 이른바 'Jev 공학(Jev Engineering)'적 설계가 절대적으로 요구된다²¹.

6.1. 문자적 직역주의와 산술 연산의 배제

Jev 모델은 텍스트를 고차원적으로 생성하는 도구가 아니라 확률에 기반한 분류기이기 때문에, 내장된 수학적 연산 능력이 전무하며 사물이나 숫자를 세는 작업(Counting)에 완전히 실패한다¹⁸. 또한 지시사항을 과도하게 문자 그대로 직역(Literal reading)하는 성향이 강해 이중 부정(Double negatives)이나 다단계 추론(Multi-hop reasoning)을 요구하는 질문을 던지면 정확도가 급락한다¹⁸. 날짜나 시간 역시 순서가 있는 정량적 수치가 아니라 단순한 문자열로 취급하여 시간 차이를 계산하지 못한다¹⁸.

- **설계적 보완 (Mitigation):** 행동 트리에서 Jev를 호출하기 전, 필요한 모든 공간적, 수학적 계산을 C++/C# 언어 레벨에서 사전에 완료해야 한다²⁰. NPC와 플레이어 간의 육각 격자 거리 좌표나 체력의 백분율 증감, 탄약의 절대 수치를 모델에게 계산하라고 요구해서는 안 된다. 클라이언트가 거리와 각도를 모두 정수로 변환하여 `{"distance_to_player": 4, "is_reloading": true, "health_percent": 15}`와 같이 완전히 정제된 정량적 상태(State) 텍스트를 Jev에 주입하고, 모델에게는 오로지 "이 상황에서 가장 합리적인 생존 본능적 판단은 무엇인가?"라는 1차원적 직관(Intuition)만을 물어야 완벽하게 작동한다²⁰.

6.2. 상태 과부하 및 맥락 부패(Context Rot)의 통제

Jev는 64,000 토큰에 달하는 넉넉한 컨텍스트 윈도우(Context Window)를 지원하지만, 입력 상태(State) 내에 해당 질문과 무관한 배경 설정이나 노이즈 텍스트가 과도하게 섞여 있을 경우 주의력 메커니즘이 분산되어 결정의 정확도가 크게 하락하는 현상을 겪는다¹³.

- **설계적 보완 (Mitigation):** 게임 월드 내의 방대한 로어(Lore)나 모든 이벤트 기록을 통째로 전송하는 우를 범해서는 안 된다. 인지 모듈이 수집한 수백 개의 가시거리 내 오브젝트 중 해당 NPC의 현재 목표와 가장 직결된 필터링된 정보만을 골라내는 경량화 파이프라인이 필수적이다. 질문(Question)이 요구하는 판단에 반드시 필요한 맥락 필드만을 선별하여 전송함으로써 입력 토큰의 잡음을 최소화해야 한다²⁰.

6.3. 적대적 환경(Adversarial Content)과 상태 불변성 관리

사용자가 직접 입력하는 채팅 텍스트 등 적대적인 문자열이 상태 객체에 주입될 경우, Jev의 결과값이 의도치 않게 편향되거나 답이 유도될 수 있는 취약점이 보고된다¹⁸. 또한 여러 질문을 동시에 처리할 때 Jev 자체는 각 답변 간의 구조적 불변성(상호 모순 없는 일관성)을 100% 보장해주지 않는다²⁰.

- 설계적 보완 (Mitigation): 사용자 입력 텍스트를 바탕으로 상태를 분류할 때는 시스템 코드 층위에서 사전 검증을 통해 프롬프트 주입 공격을 정제해야 한다. 더불어, Jev가 상충하는 두 가지 답변(예: "매우 다급함"이라는 Score 결과와 "무시함"이라는 Choice 결과)을 동시에 내놓을 가능성에 대비하여 행동 트리 내부에서 충돌 해결 규칙(Conflict Resolution Rule)을 코드로 명문화해 두어야 한다²⁰.

7. 결론

TypeSafe AI가 제시한 Jev는 화려한 문장을 작성하던 범용 챗봇 LLM의 환상에서 벗어나, 소프트웨어 스택 깊숙한 곳에서 즉각적이고 안정적인 의사결정을 지원하는 '시스템 원(System One)'이라는 새로운 지능형 인프라의 기준을 제시하고 있다⁹. 복잡한 토큰 생성 과정을 과감히 생략한 이 비자기회귀 구조는 100~500밀리초 단위의 극히 낮은 지연 시간과 프론티어 LLM 대비 400배 저렴한 압도적인 비용 절감을 실현하며, 게임 개발 환경에 딥러닝 기반 인공지능을 대규모로 배포할 수 있는 실현 가능한 경로를 열어주었다⁹.

게임의 논플레이어 캐릭터(NPC) 제어에 있어서 Jev의 활용 가치는 기호적 AI(Symbolic AI)와 신경망 AI(Neural AI)가 결합하는 신경 기호형(Neuro-Symbolic) 아키텍처의 완성으로 집약된다⁷. Jev의 세 가지 근본적 프리미티브인 Choice(범주 라우팅), Score(유틸리티 평가), Noul(안전성 검증)은 기존 행동 트리(Behavior Tree)와 유틸리티 스코어링 시스템의 경직성을 완벽하게 해소한다⁹. 언리얼 엔진의 FHttpModule과 유니티 엔진의 UnityWebRequest를 활용한 정밀한 비동기 프로그래밍 기법을 병행한다면, 게임의 메인 렌더링 프레임을 전혀 방해하지 않고도 매 순간 수십, 수백 명의 NPC가 복합적인 맥락을 인지하고 유기적으로 전술을 수정하는 메커니즘을 상용 게임 루프 내에 구축할 수 있다⁴².

다만, 이 혁신적인 도구는 결코 완벽한 만능열쇠가 아니다. 수학적 계산 능력의 부재, 과도한 문맥에 의한 판단력 부패, 문자적 직역주의와 같은 Jev의 구조적 결함(Jaggedness)은 개발자의 클라이언트 코드 작성 능력을 통해 철저히 상쇄되어야 한다²⁰. 거시적인 연산과 상태 정규화는 고전적인 게임 엔진 코드 내부에 남겨두고, 가장 직관적이고 인간적인 판단력이 요구되는 분기점에서만 Jev의 확률적 결정을 블랙보드에 주입하는 절제된 'Jev 공학적' 접근법이 요구된다.

결론적으로 Jev의 등장은, 수천 개의 노드 연결과 조건문의 조합 폭발로 인해 무너져가던 수작업 기반 NPC 설계 패러다임에 종지부를 찍고, 예측 가능하면서도 유연하게 살아 숨 쉬는 차세대 게임 세계를 창조하기 위한 핵심적인 도약대가 될 것이다.

참고 자료

1. Player Games using LLM-Generated Behavior Trees - OpenReview, <https://openreview.net/pdf?id=2uQjSKKvPa>
2. UC Merced - eScholarship.org, <https://escholarship.org/content/qt3vt6798v/qt3vt6798v.pdf>

3. Utility system - Wikipedia, https://en.wikipedia.org/wiki/Utility_system
4. Implementing Behavior Trees using Three-Valued Logic - arXiv, <https://arxiv.org/pdf/2011.03835>
5. An LLM-Based Behavior Agent with Natural Language Personality, <https://etasr.com/index.php/ETASR/article/download/12631/5504>
6. One Policy, Infinite NPCs: Persona-Traceable Shared RL ... - arXiv, <https://arxiv.org/html/2605.23652v1>
7. The Mind and the Body: A Hybrid Architecture for Believable Game AI, <https://aaltodoc.aalto.fi/bitstreams/59c4055a-625f-4ff0-bb9f-09897fcc5851/download>
8. Neuro-Symbolic Game AI: Guardrails for NPC Intelligence - Veriprajna, <https://veriprajna.com/whitepapers/neuro-symbolic-game-ai-beyond-infinite-freedom>
9. Jev (AI model) - Wikipedia, [https://en.wikipedia.org/wiki/Jev_\(AI_model\)](https://en.wikipedia.org/wiki/Jev_(AI_model))
10. Jev Explained: Typesafe AI's Non-Autoregressive System-1 Model, <https://www.mindstudio.ai/blog/jev-system-one-model-launch>
11. TypeSafe Jev Explained: The AI Model That Decides Instead of, <https://chatmaxima.com/blog/typesafe-jev-system-one-model/>
12. What Is Jev? Inside TypeSafe's Decision-Only AI Model and Its, <https://www.firecrawl.dev/blog/what-is-jev>
13. Jev API: TypeSafe's System One Model, Pricing, and Use Cases, <https://apimaster.ai/blog/jev-api>
14. TypeSafe AI: Home, <https://typesafe.ai/>
15. TypeSafe AI's Jev: What "System One Models" Actually Are, <https://www.truefoundry.com/ja/blog/typesafe-ai-jev>
16. Meet Jev, the new AI model built to make decisions instead of generating text, <https://indianexpress.com/article/technology/artificial-intelligence/meet-jev-new-ai-model-from-chatgpt-inventor-10887591/>
17. convaiinnovations/laya · Hugging Face, <https://huggingface.co/convaiinnovations/laya>
18. What Is Jev? TypeSafe AI's New Model, Explained - Crescent AI, <https://www.ai-crescent.com/blog/what-is-jev-ai-2026>
19. TypeSafe AI 'Jev' 모델의 학습 로직 및 RLCD 알고리즘에 관한 심층, https://note.com/wayne_chang/n/n151303c2041a?hl=ko
20. TypeSafe AI Releases Jev: A System One Model That Returns, <https://www.marktechpost.com/2026/09/19/typesafe-ai-releases-jev/>
21. Jev (TypeSafe) explained: the AI model that returns decisions, <https://ntorres.dev/blog/jev-typesafe-system-one-model>
22. Jev Model Explained: API Pricing, Uses & Instant Access - OmniaKey, <https://omniakey.com/blog/jev-model-explained>
23. How to Use Jev: A practical guide to TypeSafe's System One model, <https://dev.to/valyuai/how-to-use-jev-a-practical-guide-to-typesafes-system-one-model-g5e>
24. What Is Jev? A Guide to TypeSafe AI's System One Model - LangChain, <https://www.langchain.com/blog/building-a-harness-with-jev>

25. Spring AI and TypeSafe Jev: Fast, Cheap, Structured Decisions,
<https://spring.io/blog/2026/09/21/spring-ai-typesafe-structured-judgment/>
26. Getting Started with Jev in Java and Spring Boot - Dan Vega,
<https://www.danvega.dev/blog/getting-started-jev-java-spring-boot>
27. How to Use Jev: Tested Guide With Working Code (2026),
<https://www.ayautomate.com/blog/how-to-use-jev-guide>
28. Comprehensive project reference for TypeSafe Jev - GitHub Gist,
<https://gist.github.com/pjburnhill/adf8d28efcad9df037bfdece178ef965>
29. Non-player character decision-making in computer games,
<https://www.semanticscholar.org/paper/Non-player-character-decision-making-in-computer-Uluda%C4%9F%C4%B1-Oguz/5f1774f1aca24046de5f6d9eef53dd354725149f>
30. Agents Play Thousands of 3D Video Games - Zhongwen Xu,
<https://zhongwen.one/pdfs/PORTAL.pdf>
31. Game AI NPC Intelligence and Edge Inference | Veriprajna,
<https://veriprajna.com/solutions/game-ai-npc-intelligence>
32. jev: calibrated decisions for agents - GitHub Gist,
<https://gist.github.com/pedramamini/014676fa8684d91bf7000f4623701ada>
33. Jev might be the next frontier for improving roleplays : r/SillyTavernAI,
https://www.reddit.com/r/SillyTavernAI/comments/1wl7uje/jev_might_be_the_next_frontier_for_improving/
34. Meet TypeSafe AI's Decision Model: Jev - Medium,
<https://medium.com/@ajay-arunachalam08/meet-typesafe-ais-decision-model-jev-42c02f3acab9>
35. UBTService_BlackboardBase | Unreal Engine 5.8 Documentation,
https://dev.epicgames.com/documentation/unreal-engine/API/Runtime/AIModule/UBTService_BlackboardBase
36. 언리얼엔진5 c++ BService , Blackboard 변수변경,
<https://makerejoicegames.tistory.com/691>
37. Onchain Utility AI: A Litepaper - Ather Labs Atlas,
<https://atlas.atherlabs.com/atherxperiment/onchain-utility-ai>
38. AI Framework: FSM, Behavior Trees, GOAP | PDF - Scribd,
<https://www.scribd.com/document/699666891/AI-FSM-Behavior-Tree-GOAP-Utility-AI-Nez-framework-documentation>
39. What's the best approach to implement basic enemy AI? FSM vs,
https://www.reddit.com/r/unity/comments/1mj5mhq/whats_the_best_approach_to_implement_basic_enemy/
40. How to Build a Stalker System for Your Game - Medium,
https://medium.com/@armoury_ale_0s/how-to-build-a-stalker-system-for-your-game-77ad2babb127
41. Unreal Engine 4 - How to avoid blocking game thread when saving,
<https://gamedev.stackexchange.com/questions/134059/unreal-engine-4-how-to-avoid-blocking-game-thread-when-saving-file-to-storage>
42. [Unreal] [Networking] Http Module 사용 및 API Handler 개발,
<https://algorfati.tistory.com/194>

43. [UE5/C++] Unreal에서 Http 요청하기 (1) - velog,
<https://velog.io/@singery00/UE5C-Unreal%EC%97%90%EC%84%9C-Http-%EC%9A%94%EC%B2%AD%ED%95%98%EA%B8%B0-1>
44. UE4.10 How To Make HTTP GET Request in C++,
<https://unrealcommunity.wiki/ue4.10-how-to-make-http-get-request-in-cpp-ektwg52q>
45. HTTP Requests In Unreal Engine C++ - GitHub Gist,
<https://gist.github.com/yexiuph/d85ead762a626e5c2e8be890959d80b6>
46. FHttpmodule threading : r/unrealengine - Reddit,
https://www.reddit.com/r/unrealengine/comments/1o64a5v/fhttpmodule_threading/
47. 언리얼엔진 C++ Thread 만들기 (FRunnable) - online5880 - 티스토리,
<https://online-unreal.tistory.com/entry/%EC%96%B8%EB%A6%AC%EC%96%BC%EC%97%94%EC%A7%84-C-Thread-%EB%A7%8C%EB%93%A4%EA%B8%B0-FRunnable>
48. Weird problem with Async Task and UnityWebRequest,
<https://discussions.unity.com/t/weird-problem-with-async-task-and-unitywebrequest/226361>
49. Fix: Unity Async Await Freezing or Blocking the Main Thread - Bugnet,
<https://bugnet.io/blog/fix-unity-async-await-freezing-main-thread>
50. Async in Unity: Main Thread First, Multithread When Needed,
<https://daily.dev/posts/async-in-unity-main-thread-first-multithread-when-needed-gcoeldrxj>
51. Why await Resumes on the Main Thread in Unity,
<https://discussions.unity.com/t/why-await-resumes-on-the-main-thread-in-unity-synchronizationcontext/1700147>
52. Scripting API: UnityWebRequestAsyncOperation - Unity - Manual,
<https://docs.unity3d.com/6000.5/Documentation/ScriptReference/Networking.UnityWebRequestAsyncOperation.html>
53. UnityWebRequest를 구현하는 다양한 방법들 스터디 · GitHub,
<https://github.com/Joseph-Cha/UnityWebRequestPractice>
54. Async in Unity: Main Thread First, Multithread When Needed,
<https://www.youtube.com/watch?v=GSjBWnfBzbQ>
55. Await support - Unity - Manual,
<https://docs.unity3d.com/2023.2/Documentation/Manual/AwaitSupport.html>
56. Using the Blackboard in Unity Behavior | Unity Tutorial - YouTube,
<https://www.youtube.com/watch?v=YWGYv95gSfY>
57. Behavior Tree (4) - 데이터 전달하기(Blackboard) - 게임 클라 개발,
<https://tsyang.tistory.com/126>
58. Behavior Tree BlackboardVariableValue changes references,
<https://discussions.unity.com/t/behavior-tree-blackboardvariable-t-value-changes-references-despite-editor-assignment/1542617>
59. Jev, the AI That Never Writes a Sentence: Cheaper and Faster Than,
<https://xenospectrum.com/en/jev-typesafe-bert-classifier-decomposition/>